Building an MCP Server — 3-Hour Class

A demo-driven deep dive into the Model Context Protocol — short concept segments anchoring long live IDE demonstrations of every chapter lesson, from JSON-RPC transport to a supervised AI agent.

How This Class Runs

This is not a slides-first presentation. The deck anchors short concept segments; the bulk of the class happens **live in the IDE**, walking through each chapter of the workshop lessons on the finished codebase. Each chapter follows the same rhythm:

1. Concept (slides)

A few minutes on this page's cards — what the chapter adds to the protocol and why it exists.

2. Demo (IDE)

Switch to the IDE. Walk the lesson's `lessons/NN-instructions.md` against the real code in `src/main/java/com/workshop/mcp/`.

3. Verify (Inspector)

Exercise the new capability in MCP Inspector — watch the JSON-RPC messages flow both ways.

The deck flows left to right with no fixed timings — pace each segment to the room and its questions. Keep Runtime Facts and Trace Log open in tabs as reference during demos.

📐

STOP 1

Protocol Overview

MCP architecture, actors (User → App → LLM → MCP Server), key protocol features, and why MCP exists.

⚡

STOP 2 · CH 1–2

Transport & Protocol + Ch 1 Demo

STDIO transport layer, IOHandler implementation, JSON-RPC 2.0 message types — then a long IDE demo through the Chapter 1 transport code.

📊

STOP 3 · CH 2

Flow Diagram

Interactive sequence diagram — every JSON-RPC message from initialize through tools/call. Hover messages to see full JSON payloads.

🔧

STOP 4 · CH 3–4

Handshake & Capabilities + Ch 3–4 Demos

Chapter 3 handshake and routing with the first Inspector connect, then Resources, Tools, and Prompts — a two-part demo.

🧩

STOP 5 · CH 5

Extensions + Ch 5 Demo

The experimental capability map, MCP extension identifiers, and the full lazy Elicitation flow demonstrated live in Inspector.

🖥️

STOP 6 · CH 6

MCP Apps + Ch 6 Demo

Interactive HTML UI rendered inside the conversation — Tool + Resource + _meta pattern, then the dashboard rendered live in the Inspector Apps tab.

🧵

STOP 7 · CH 7

Tasks + Ch 7 Demo 2025-11-25

Call-now / fetch-later via task augmentation. CreateTaskResult, tasks/get, tasks/result, tasks/list, tasks/cancel — demonstrated end to end.

🤖

STOP 8 · AGENT

Supervised Agent + Demo

Claude Code plugin with four composable skills and a supervised 6-step audit workflow with approval checkpoint — run live.

📋

REFERENCE

Runtime Facts

Protocol version, client/server capabilities negotiated, available tools, session flow — key reference during IDE demos.

🔍

REFERENCE

Trace Log

Raw JSON-RPC communication trace with timestamps. Filter by sent/received. Toggle formatted JSON. Compare against live Inspector output.

What You'll See Built

By the end of this class, you will have watched a complete Java MCP server demonstrated layer by layer:

- **Speaks JSON-RPC 2.0** over STDIO with full bidirectional communication
- **Exposes Resources** — 70+ Javadoc HTML files served from the JAR classpath
- **Provides Tools** — a `key_word_search` tool with JSON Schema parameters
- **Offers Prompts** — user-friendly templates that guide clients toward tool usage
- **Supports Elicitation** — server-initiated user input collection via forms
- **Renders MCP Apps** — an interactive search dashboard inside the conversation
- **Handles Tasks** — asynchronous call-now / fetch-later tool execution
- **Powers an Agent** — a Claude Code plugin that runs a supervised keyword audit workflow

Prerequisites: JDK 21+, Gradle, Node.js (for MCP Inspector)

MCP Protocol Overview

Understanding the Model Context Protocol — architecture, actors, and communication flow

What is the Model Context Protocol?

MCP is an **open standard** that defines how AI applications, language models, and external services communicate. Think of it as the USB-C of AI integrations — a universal connector that lets any LLM-based application speak to any data source or tool without custom glue code.

MCP uses **JSON-RPC 2.0** as its message format, runs over STDIO (and HTTP/SSE), and enables bidirectional communication where both the client and server can initiate requests.

The Four Actors

👤 **User**
The human who initiates a request — types a question or command in the chat interface.

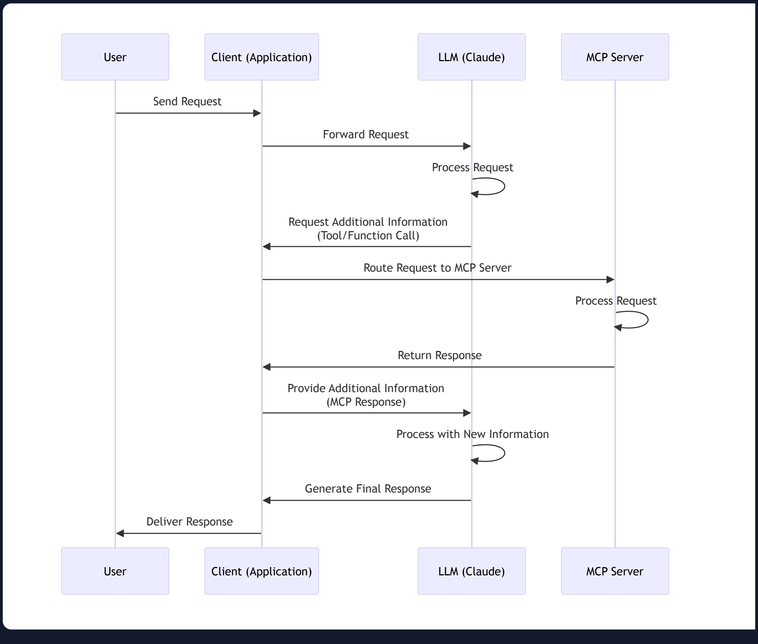
📱 **Client (Application)**
Claude Desktop, Claude Code, VS Code Copilot, Cursor — the client that owns the conversation and manages MCP connections.

🧠 **LLM**
The language model (e.g., Claude) that processes requests, reasons about them, and decides when to call tools.

🔌 **MCP Server**
Your code. Exposes tools, resources, and prompts that the LLM can discover and invoke. What we're building today.

MCP Communication Architecture

The diagram below shows how a complete request-response cycle flows through all four actors.



Flow Explanation

1. User Interaction

The process begins when a `User` sends a request to the `Application`. This could be a question, command, or any input that requires processing.

2. LLM Processing

The `Application` forwards the request to the `LLM`, which processes the user's input. During processing, the LLM may determine it needs external capabilities to give a complete answer.

3. Tool/Function Call

When the `LLM` requires external data or functionality, it makes a `Tool/Function Call` back to the Application. This is the MCP entry point — the LLM is requesting access to a registered tool or resource.

4. MCP Server Interaction

The `Application` routes this request to the appropriate `MCP Server`. The server processes the request, which might involve:

- Accessing databases or file systems
- Calling external APIs
- Performing computations
- Retrieving specific resources or documentation

5. Response Chain

The `MCP Server` returns its response to the Application, which provides this additional information back to the LLM. The LLM processes this new information and generates a comprehensive final response.

6. Final Delivery

The `Application` delivers the LLM's final response back to the `User`, completing the request-response cycle.

Key Protocol Features

🔌 **Transport Agnostic**
Works over STDIO, Server-Sent Events (SSE), and Streamable HTTP. Protocol logic is independent of transport.

↔️ **Bidirectional**
Both client and server can initiate requests. Enables real-time updates, sampling, elicitation, and root change notifications.

🧩 **Extensible**
New capabilities can be added via the `experimental` map without breaking existing implementations.

📋 **Capability Negotiation**
Both sides declare what they support during `initialize`. No guessing — opt-in features only.

🔧 **Tools**
Functions the LLM can call. Defined with JSON Schema so clients can validate parameters before sending.

📄 **Resources**
Data sources the client can discover and read. URIs + MIME types. Supports pagination with cursors.

💬 **Prompts**
Reusable prompt templates that guide clients in composing effective tool calls.

🤝 **Sampling**
Server can ask the client to run an LLM inference on its behalf — for AI-powered server logic.

Why Build an MCP Server in Java?

- **Existing codebases:** Most enterprise Java code can be wrapped and exposed as MCP tools with minimal change
- **Type safety:** Java's type system maps naturally to JSON Schema for tool parameters
- **JVM ecosystem:** Access to the full Maven/Gradle ecosystem for any integration you need
- **One server, many clients:** Claude Code, Cursor, Windsurf, VS Code Copilot — any MCP-compatible host can use your server

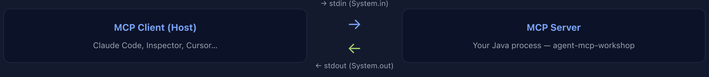
Transport Layer & JSON-RPC Protocol

How MCP moves messages: STDIO transport, IOHandler architecture, and JSON-RPC 2.0 message types

Chapter 1

Transport Layer — The STDIO Foundation

MCP communicates over **standard I/O streams** by default. The host process starts your server as a subprocess and communicates through stdin/stdout. Every message is a single JSON line.



Critical Rule: Never write anything to `System.out` except JSON-RPC messages. All debugging and logging must go to a file. Writing non-JSON to stdout breaks the protocol connection immediately.

IOHandler — The Communication Hub

The `IOHandlerImpl` class manages all I/O with three responsibilities:

Input Reading

```
public void startInputReader() {
    Scanner scanner = new Scanner(System.in);
    running.set(true);
    while (running.get()) {
        if (!scanner.hasNextLine()) {
            running.set(false); break;
        }
        String line = scanner.nextLine();
        logger.log("[API][RECEIVED]" + line);
        publishLine(line); // → listeners
    }
}
```

A blocking read loop — `Server.start()` calls it on the main thread after registering the router as a listener. Publishes each JSON line to all registered listeners via the Observer pattern.

Output Emission

```
public void emit(Object message) {
    String text = gson.toJson(message);
    logger.log("[API][SENT]: " + text);
    writer.println(text);
    writer.flush(); // immediate
}
```

Serializes any Java object to JSON via Gson. Immediately flushes — MCP clients expect real-time responses.

Key Design Choices

- **CopyOnWriteArrayList** for thread-safe listener management
- **AtomicBoolean** for safe running-state across threads
- **Process-specific log files** using PID — no conflicts when multiple servers run
- **Router interface** — the IOHandler doesn't know what messages mean, it just moves them

Chapter 2

JSON-RPC 2.0 — The Message Protocol

MCP uses **JSON-RPC 2.0** as its wire format. Every message is a JSON object with a defined structure. There are four message types:

REQUEST

Client or Server → Other Side

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "method": "tools/list",
  "params": {
    "_meta": { "progressToken": 6 }
  }
}
```

Has an `id` — expects a Response back with the same id.

RESPONSE

Other Side → Requester

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "result": {
    "tools": [{
      "name": "key_word_search",
      "description": "..."
    }]
  }
}
```

Returns `result` with the same `id` as the Request.

NOTIFICATION

Either Side → Other Side

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

No `id` — fire-and-forget. No Response expected.

ERROR RESPONSE

Server → Client (on failure)

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "error": {
    "code": -32601,
    "message": "Method not found"
  }
}
```

Standard error codes: -32700 Parse Error, -32600 Invalid Request, -32601 Method Not Found.

MCP Method Namespace

All MCP methods follow a `category/action` naming convention:

INITIALIZE	Protocol handshake — first message, always
RESOURCES/*	list, read — discover and fetch content
TOOLS/*	list, call — discover and execute tools
PROMPTS/*	list, get — discover and fetch templates
SAMPLING/*	createMessage — server asks client to run LLM
ELICITATION/*	create — server requests structured user input

Stateless by Design

Each request-response pair is independent. The server holds no session state between calls. This means:

- Any request can arrive in any order (after initialization)
- Servers can be scaled horizontally
- Debugging is simpler — each message is self-contained

Capability-Based Feature Detection

During `initialize`, both sides declare what they support. The server only sends elicitation requests if the client advertised `elicitation: {}`. The client only uses sampling if the server didn't advertise it. No guessing — explicit opt-in at every level.

IDE DEMO — CHAPTER 1

Chapter 1 Live: The Transport Layer in Code

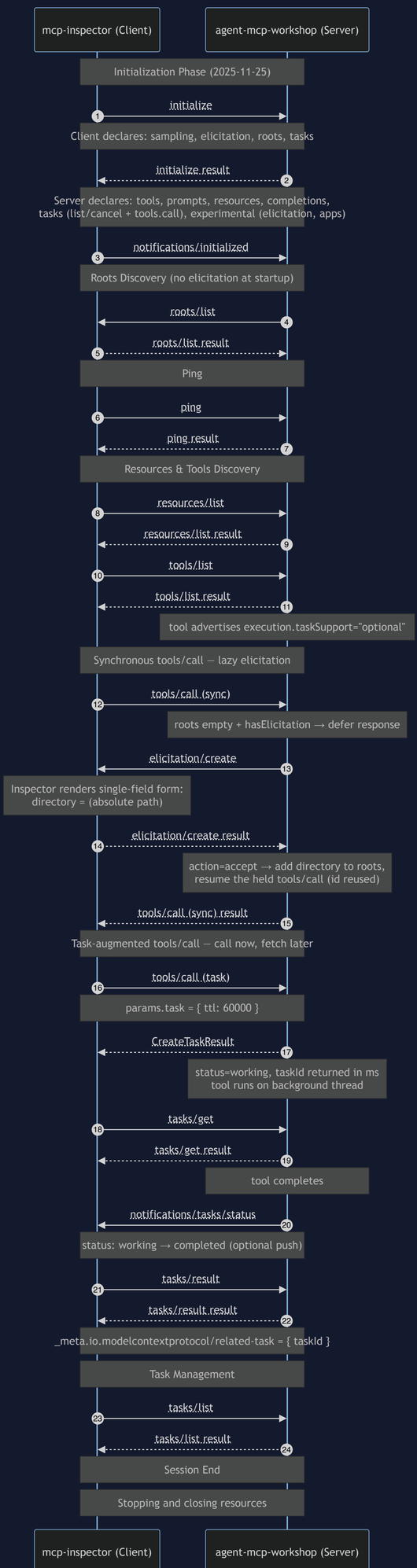
Switch to the IDE. Have `lessons/01-instructions.md` open in a side tab — it is the script for this demo. You are on the `complete` branch, so everything already works; narrate each block as the lesson presents it.

1. **Open** `src/main/java/com/workshop/mcp/io/IOHandlerImpl.java` — this class is the entire transport layer. Point out the collaborators at the top: the auto-flush `PrintWriter` over System.out, the `running` AtomicBoolean, the `lineListeners` CopyOnWriteArrayList, and the Gson instance.
2. **Walk** `startInputReader()` — the Scanner loop over `System.in`. Trace the happy path: `hasNextLine()` → `nextLine()` → log `[API][RECEIVED]` → `publishLine(line)`. Then trace the shutdown path: stream closes → `running.set(false)` → `stopRunning()` in the finally block.
3. **Walk** `publishLine()` — the observer pattern in six lines. Each listener call is wrapped in its own try-catch so one faulty listener never breaks the others. The router registers itself as a listener; the transport never knows what messages mean.
4. **Walk** `emit()` — `gson.toJson()`, log `[API][SENT]`, `println`, explicit `flush()`. Emphasize the flush: MCP clients expect real-time line-delimited responses.
5. **Open** `LogFileWriter.java` — show that all logging goes to a PID-named file, never stdout. Reinforce the critical rule from the card above: stdout is protocol-only.
6. **Build and test:** in the terminal run `./gradlew clean build`. Open `IOHandlerImplTest.java` and show the tests exercising the reader loop and emit path passing.
7. **Prove it end to end:** run `java -jar build/libs/agent-mcp-workshop-0.0.1.jar`, paste a raw JSON-RPC line (e.g. a ping request) into stdin, and show the JSON response come back on stdout plus both `[API][RECEIVED]` and `[API][SENT]` entries in the log file.

What the audience should observe: the transport is dumb on purpose — it moves lines and knows nothing about MCP. Everything protocol-aware lives behind the listener boundary. Next we look at what those lines contain: JSON-RPC 2.0, then the full session flow on the Flow Diagram.

MCP Presentation – JSON-RPC Communication Flow

Sequence diagram showing the complete MCP communication flow including all phases. Hover message labels to see full JSON payloads.



💡 Hover or tap message labels to see full JSON payloads. This diagram shows the complete MCP communication lifecycle from initialization to shutdown.

⌨️ UP NEXT: IDE DEMO — CHAPTER 3

Chapter 3: Bring the Handshake to Life

The first three arrows of this diagram — **initialize → initialize result → notifications/initialized** — are exactly what the Chapter 3 demo makes real. Switch to the IDE and follow the full script on the Capabilities page: walk the INITIALIZE and PING handlers in `IORouter`, launch MCP Inspector, and watch these messages appear live. Keep the Trace Log open to compare against Inspector's message pane.

Capabilities: Resources, Tools & Prompts

Implementing the three core MCP capabilities that transform a basic router into a fully functional server

Chapter 3

Protocol Handshake & Routing

Before any capability is usable, the client and server must complete the **initialize handshake**. This negotiates protocol version and declares what each side supports.

Initialize Request (Client → Server)

```
{
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
      "sampling": {},
      "elicitation": {},
      "roots": { "listChanged": true }
    },
    "clientInfo": {
      "name": "mcp-inspector",
      "version": "0.16.5"
    }
  }
}
```

Initialize Response (Server → Client)

```
{
  "result": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
      "tools": { "listChanged": true },
      "prompts": { "listChanged": true },
      "resources": {
        "listChanged": false,
        "subscribe": false
      }
    },
    "serverInfo": {
      "name": "agent-mcp-workshop",
      "version": "0.0.1"
    }
  }
}
```

Message Routing — The IORouter Switch

After deserialization, every message is routed through a `switch` expression based on its method name:

```
case INITIALIZE       → handleInitialize(request)
case PING             → emit(new JsonRpcResponse(id, new Object()))
case RESOURCES_LIST   → handleResourcesList(request)
case RESOURCES_READ   → handleResourceRead(request)
case TOOLS_LIST       → handleToolsList(request)
case TOOLS_CALL       → handleToolsCall(request)
case PROMPTS_LIST     → handlePromptsList(request)
case PROMPTS_GET      → handlePromptGet(request)
case NOTIFICATIONS_*  → handleNotification(notification)
```

STDIO Discipline

- All logs go to `logs/agent-mcp-workshop-<pid>-<timestamp>.log` (relative to the server's working directory — `inspector/logs/` when launched by the Inspector) — never to System.out
- Writing non-JSON to stdout breaks the protocol immediately
- The client reads stdout byte-by-byte looking for newline-delimited JSON

IDE DEMO — CHAPTER 3

Chapter 3 Live: Handshake, Routing & First Inspector Connect

Switch to the IDE with `lessons/03-instructions.md` open as the script. This is the first time the class sees the server talk to a real client.

1. **Open** `src/main/java/com/workshop/mcp/IORouter.java` — walk the `INITIALIZE` case: deserialize `InitializeParams`, check client capabilities (`roots`, `sampling`, `elicitation`, `tasks`), then build the response with `InitializeResultBuilder`. Note: on the `complete` branch the builder chain already declares the later chapters too (experimental elicitation/apps, tasks) — the Ch 3 lesson version is just protocol version + default capabilities + server info.
2. **Show the `PING` case** — two lines: `success(message.id(), new Object())`. Explain request-response correlation via the message id.
3. **Build:** `./gradlew clean build` in the project root terminal.
4. **Launch MCP Inspector:** show `inspector/config.json` (how the Inspector launches the JAR), then `cd inspector && ./run.sh`. Open the printed URL in the browser.
5. **Connect** — watch the green status. In the message pane, show the live `initialize → initialize result → notifications/initialized` exchange, matching the first arrows of the Flow Diagram.
6. **Click Ping** — the empty-object response comes back with the matching id. Heads-up: on the `complete` branch, Ping also fires a demo `sampling/createMessage` request at the client when it advertised sampling — a bonus teaching moment if Inspector shows it.
7. **Teaching moment — cancellation:** on the plain Ch 3 build, clicking "List Resources" would time out because the capability is advertised but unimplemented. Show the `NOTIFICATION_CANCELLED` case and `NotificationCancelledParams` record in `IORouter` / `spec` — the client's cancellation reason gets deserialized into a typed record and logged.
8. **Open the log file** in `inspector/logs/` — show `[API][RECEIVED]` / `[API][SENT]` pairs for the whole session. Reinforce: logs to file, protocol to stdout.

What the audience should observe: capability advertisement is a contract — the client immediately exercises whatever you claim to support. Leave the Inspector running; the next demo builds on this session.

Chapter 4

📄 Resources — Serving Content from the Server

Resources allow the server to expose any content the client can discover and read. Our server exposes **70+ Javadoc HTML files** bundled inside the JAR.

resources/list — Discovery

Returns a paginated list of all available resources with URI, name, description, and MIME type:

```
{
  "resources": [{
    "uri": "javadoc/com/workshop/mcp/spec/Capability.html",
    "name": "Capability",
    "description": "Represents the capability...",
    "mimeType": "text/html"
  }],
  "nextCursor": "pageNext"
}
```

resources/read — Retrieval

Returns the actual content of a specific resource by URI:

```
{
  "contents": [{
    "uri": "javadoc/...",
    "mimeType": "text/html",
    "text": "..."
  }],
  "isError": false
}
```

JavadocResources — Classpath Loading

- HTML Javadoc files are bundled inside the JAR via Gradle resources
- **JSoup** parses the HTML to extract documentation summaries for descriptions
- File paths become resource URIs: `javadoc/com/workshop/mcp/spec/Capability.html`
- Making the server self-contained — no external files required

🔧 Tools — Executable Functionality

Tools are functions the LLM can call. Defined with **JSON Schema** for parameter validation, they are the primary integration point between the LLM and your business logic.

tools/list — Discovery

```
{
  "tools": [{
    "name": "key_word_search",
    "description": "Searches for a keyword across all files in a project.",
    "inputSchema": {
      "type": "object",
      "properties": {
        "keyword": {
          "type": "string",
          "description": "keyword to search for"
        }
      },
      "required": ["keyword"]
    },
    "execution": { "taskSupport": "optional" }
  }]
}
```

tools/call — Execution

```
{
  "method": "tools/call",
  "params": {
    "name": "key_word_search",
    "arguments": {
      "keyword": "mcp"
    }
  }
}

// Response
{
  "content": [{
    "type": "text",
    "text": "/path/api.http, keyword_count=1"
  }],
  // ... 57 more files
  ],
  "isError": false
}
```

The directory is **not** a tool param — the server gets it from `roots/list` or the elicitation form. On the `complete` branch the entry also carries the Ch 6 `_meta.ui.resourceUri` block.

KeyWordSearch Implementation

- Walks the directory tree recursively from each configured root
- Roots are populated via `roots/list` or, if absent, via the lazy elicitation in Ch 5 — never as a tool parameter
- Counts occurrences of `keyword` per file (case-insensitive)
- Returns one text content item per matching file with its count
- The JSON Schema `required` array enables client-side validation before the request is even sent

IDE DEMO — CHAPTER 4 · PART 1 OF 2

Chapter 4 Live, Part 1: Resources and the key_word_search Tool

Continue in the IDE with `lessons/04-instructions.md` open. Inspector should still be connected from the Chapter 3 demo.

1. **Open** `src/main/java/com/workshop/mcp/resources/JavadocResources.java` — walk the classpath-loading strategy: `loadAllHtmlResourcesFromFolder()` reads the build-time `html-files.txt` listing, `readResourceContent()` streams files from inside the JAR, and JSoup extracts descriptions from the Javadoc HTML.
2. **Back in `IORouter.java`**, walk the `RESOURCES_LIST` case (builder + `withNextCursor("pageNext")` — pagination advertised but intentionally not implemented) and the `RESOURCES_READ` case (URI validation, content read, error path with `asError()`). On `complete` both cases also carry the Ch 6 MCP App resource (`ui://keyword-search/mcp-app.html`) — mention it and move on.
3. **In Inspector:** click **List Resources** — the request that failed in the Ch 3 demo now returns 70+ Javadoc files. Click one and show the HTML coming back via `resources/read`.
4. **Open** `src/main/java/com/workshop/mcp/tools/KeyWordSearch.java` — show `name()`, `description()`, and `schema()`: a single required `keyword` parameter. Emphasize the design decision: *the search directory is not a tool argument — it comes from MCP roots* (or, in Ch 5, elicitation).
5. **Walk the `TOOLS_LIST` and `TOOLS_CALL` cases in `IORouter`** — name matching, deserializing `ToolCallParams`, and the "Tool not found" error path. Two `complete`-branch details worth pointing out: `TOOLS_LIST` returns the Ch 6 `AppTool` (with `_meta` and `execution`) and it *clears the roots set* each time it runs — a deliberate workshop trick so the Ch 5 elicitation is always demonstrable.
6. **In Inspector:** click **List Tools**, inspect the JSON Schema, then call `key_word_search` with keyword `"mcp"`. Because listing tools just cleared the roots, the Ch 5 elicitation form will pop up asking for a search directory — accept it with the project's absolute path (tell the class this is a preview of the Extensions chapter), and dozens of files return with occurrence counts.

What the audience should observe: the same listtread (or list/call) shape repeats for every capability — discovery first, execution second. Prompts complete the trio next. This is also a natural pause point if you want to take a break — leave the server and Inspector running.

Chapter 4

💬 Prompts — User-Friendly Templates

Prompts are reusable templates that guide users and clients in composing effective tool calls. They appear as autocomplete suggestions in MCP-compatible clients.

prompts/list — Discovery

```
{
  "prompts": [{
    "name": "search_keyword",
    "description": "Creates a prompt to search for a word using the key_word_search tool.",
    "arguments": [{
      "name": "keyword",
      "description": "The word to search for",
      "required": true
    }]
  }]
}
```

prompts/get — Execution

```
{
  "method": "prompts/get",
  "params": {
    "name": "search_keyword",
    "arguments": { "keyword": "mcp" }
  }
}

// Response
{
  "messages": [{
    "role": "user",
    "content": {
      "type": "text",
      "text": "Start from the root of the project and use the key_word_search tool to search for this specific keyword: "mcp""
    }
  }]
}
```

How the Three Capabilities Work Together

Capability	What It Exposes	Primary Use
Resources	Static content (Javadoc HTML)	Documentation, reference data, context for the LLM
Tools	Executable functions (keyword search)	Dynamic operations, data retrieval, system interaction
Prompts	Guided templates	UX layer — makes tools discoverable and easy to use

IDE DEMO — CHAPTER 4 · PART 2 OF 2

Chapter 4 Live, Part 2: Prompts and the Full Capability Tour

Reconnect Inspector if needed (`cd inspector && ./run.sh`), then pick up `lessons/04-instructions.md` at Part 3.

1. **Walk the `PROMPTS_LIST` case** in `IORouter.java` — the `search_keyword` prompt declares a required `keyword` argument and its description names the tool it drives.
2. **Walk `PROMPTS_GET`** — how the prompt arguments are folded into the `KEY_WORD_MESSAGE` template that guides the client toward calling `key_word_search`.
3. **Show `COMPLETION_COMPLETE`** — the autocomplete handler returning suggested values (`java`, `the`, `and`) for the keyword argument. This is what powers argument autocomplete in MCP clients.
4. **In Inspector:** click **List Prompts** → select `search_keyword` → type a keyword and watch autocomplete suggestions arrive → execute the prompt and show the generated user message.
5. **Full tour:** run the trio end to end — list resources, read one; list tools, call `key_word_search`; list prompts, get one. Narrate the repeating discover/execution shape.
6. **Wrap with references:** pivot Runtime Facts on screen for the negotiated session summary, and Trace Log to compare the recorded session against what Inspector just showed.

What the audience should observe: Tools define functionality, Prompts make it discoverable and guided. With the core server complete, everything that follows — Extensions, Apps, Tasks — is additive.

MCP Extensions & Elicitation

The experimental capability map — how MCP evolves without breaking existing implementations

Chapter 5

What Are MCP Extensions?

MCP Extensions are **optional additions** to the core specification. They allow the protocol to evolve without breaking backwards compatibility. An extension is only active when *both sides* explicitly declare support for it.

The Experimental Map: Extensions live in the `experimental` field of `ServerCapabilities`. It's a `Map<String, Object>` where each key is an extension identifier and each value is extension-specific configuration.

```
{
  "protocolVersion": "2025-11-25",
  "capabilities": {
    "tools": { "listChanged": true },
    "prompts": { "listChanged": true },
    "resources": { "listChanged": false },
    "experimental": {
      "io.modelcontextprotocol/elicitation": {},
      "io.modelcontextprotocol/apps": {}
    }
  }
}
```

Extension Identifier Format

Every extension uses a **namespaced identifier** to avoid naming collisions:

```
{vendor-prefix}/{extension-name}
```

Official MCP Extensions

Maintained by the MCP organization under the `io.modelcontextprotocol` namespace:

- `io.modelcontextprotocol/elicitation`
- `io.modelcontextprotocol/oauth-client-credentials`
- `io.modelcontextprotocol/enterprise-managed-authorization`
- `io.modelcontextprotocol/apps`

Your Own Extensions

Use your organization's reverse-domain prefix:

- `com.mycompany/analytics-dashboard`
- `com.mycompany/audit-trail`
- `io.github.username/custom-ext`

Any MCP client that doesn't recognize your extension key simply ignores it. Neither side breaks.

The Two Official Extension Repositories

🔐 ext-auth — Authorization

Covers authorization scenarios the core protocol doesn't address:

- **OAuth 2.0 Client Credentials** — machine-to-machine auth for servers calling protected APIs on their own behalf
- **Enterprise-Managed Authorization** — centralized access control via corporate identity providers (Okta, Azure AD, etc.)

🖥️ ext-apps — Interactive UI

Allows MCP servers to render interactive HTML UI elements inside the conversation:

- Charts and data visualizations
- Forms for structured input collection
- Video and media players
- Custom interactive components

This is what powers **MCP Apps** — covered on the next page.

Elicitation — Server-Initiated User Input

Elicitation lets the **server** ask the **user** for structured input mid-session. Instead of the tool requiring all parameters upfront, the server can pause execution, present a form, collect responses, and continue.

The Elicitation Flow

1. **Client declares support**
   During `initialize`, the client includes `"elicitation": {}` in its capabilities.
2. **Server checks and declares**
   Server reads `clientCapabilities.elicitation()`. If present, sets `hasElicitation = true` and adds `io.modelcontextprotocol/elicitation` to the experimental map in its response.
3. **Server sends elicitation/create — *lazily***
   The server does **not** elicit at startup. It defers the elicitation until it actually needs the missing piece — here, the first `tools/call key_word_search` arriving with an empty roots set. The original `tools/call` response is paused (the requestor's request id is remembered) while the form goes out.
4. **Client presents form to user**
   The host renders a UI form based on the schema. The user fills it in and submits.
5. **Client returns result, server resumes the deferred call**
   The form result arrives as a `JsonRpcResponse` with `id == ELICITATION_REQUEST_ID`. The server adds `content.directory` to its roots set, then sends the **delayed** response to the original `tools/call` using the request id it remembered in step 3.

Elicitation Request (Server → Client)

The workshop's `key_word_search` tool needs a directory to search in. If the client did not supply one via `roots/list`, the server asks the user directly:

```
{
  "jsonrpc": "2.0",
  "id": -4000,
  "method": "elicitation/create",
  "params": {
    "message": "Pick a directory for the keyword-search tool to search in:",
    "requestedSchema": {
      "type": "object",
      "required": ["directory"],
      "properties": {
        "directory": {
          "title": "Search Directory",
          "type": "string",
          "description": "Absolute path to the directory the keyword search should look in"
        }
      }
    }
  }
}
```

User Response (Client → Server)

```
{
  "jsonrpc": "2.0",
  "id": -4000,
  "result": {
    "action": "accept",
    "content": {
      "directory": "/Users/davidparry/code/github/agent-command"
    }
  }
}
```

The server's response handler adds `content.directory` to its `roots` set so subsequent `tools/call` invocations have a search target — without ever exposing the directory as a tool parameter.

Backwards Compatibility — The Core Principle

Direction	Mechanism	Where Implemented
Client → Server	`capabilities.elicitation: {}` in initialize request	Client (Inspector, Claude)
Server → Client	`capabilities.experimental["io.modelcontextprotocol/elicitation"]`	IORouter INITIALIZE handler
Server sends form	`elicitation/create` deferred until `tools/call` needs a directory	IORouter TOOLS_CALL branch
Client responds	`JsonRpcResponse` with id `-4000` — server adds directory to roots and resumes the held `tools/call`	IORouter process(JsonRpcResponse)

IDE DEMO — CHAPTER 5

Chapter 5 Live: The Lazy Elicitation Flow

Switch to the IDE with `lessons/05-instructions.md` open as the script.

1. **Open** `ServerCapabilities.java` in `src/main/java/com/workshop/mcp/spec/` — show the `experimental` map field, and `withExperimentalCapability()` in `InitializeResultBuilder`. Note the ordering rule: it must be chained *before* `withDefaultCapabilities()`.
2. **In** `IORouter.java`, show the `hasElicitation` flag set from `clientCapabilities.elicitation()` during INITIALIZE, and the `io.modelcontextprotocol/elicitation` declaration in the builder chain.
3. **Walk the lazy trigger** — the `TOOLS_CALL` branch: when `roots` is empty and the client supports elicitation, the router remembers `pendingToolsCallRequestId` and sends `elicitation/create` via `sendElicitationMessage()` instead of answering.
4. **Walk the resume path** in `process(JsonRpcResponse)` — id `-4000` matches, the accepted `directory` is added to roots, and the held `tools/call` is executed with its original id.
5. **In Inspector:** disconnect/reconnect for a clean session. Connect — nothing is asked at startup; only `roots/list` goes out. You don't need to clear roots by hand: on the `complete` branch, `TOOLS_LIST` clears the roots set every time it runs, so clicking **List Tools** guarantees the elicitation path.
6. **Call** `key_word_search` with a keyword — the single-field **Search Directory** form appears, and point out the tool call response has *not* arrived yet.
7. **Submit an absolute path** — the deferred `tools/call` response arrives carrying the search hits. Show the initialize response's `experimental` map and the server log entries for the whole exchange.

What the audience should observe: lazy beats eager — the form only appears when the tool actually needs the directory. The same four-step declare/detect/send/handle pattern applies to every MCP extension, including the Apps extension up next.

MCP Apps — Interactive UI Inside the Conversation

How MCP servers render live, interactive dashboards directly inside Claude — no new tab, no context switch

Chapter 6

What Are MCP Apps?

Text responses can only go so far. MCP Apps let your server return an **interactive HTML interface** that renders directly inside the conversation — right next to the chat that triggered it.

❌ Without MCP Apps

User asks → LLM calls tool → Returns wall of text:

```
/path/api.http, keyword_count=1
/path/README.md, keyword_count=3
/path/pom.xml, keyword_count=2
... 55 more files
```

Hard to explore, no interactivity, no visualization

✅ With MCP Apps

User asks → LLM calls tool → Interactive dashboard appears inline:

🔍 Keyword Search Results

Found 58 files • 400+ matches

api.http ×1 | README.md ×3 | pom.xml ×2

[Search Again] [Filter] [Export]

Sortable, filterable, interactive — stays in context

The Core Pattern: Tool + Resource = App

The tool continues to work exactly as before. The only addition is a `_meta` field in the tool definition that points the host to an HTML resource to render alongside the tool result.

What You Add to the Protocol

1. Declare the extension in initialize

```
{
  "capabilities": {
    "experimental": {
      "io.modelcontextprotocol/apps": {}
    }
  }
}
```

The host checks this field during the handshake. If absent, the app never appears — the tool still works normally.

2. Add _meta to your tool definition

```
{
  "name": "key_word_search",
  "description": "Searches for a keyword across all project files.",
  "inputSchema": { "...": "..." },
  "_meta": {
    "ui": {
      "resourceUri": "ui://keyword-search/mcp-app.html"
    }
  }
}
```

3. Serve the HTML when the host requests it

```
// In your resources/read handler:
if (uri.startsWith("ui://")) {
    // Return the bundled HTML
    return new ResourceContent(uri,
        "application/vnd.mcp-ui.app+html",
        readHtmlFromClasspath("mcp-app.html"));
}
```

The MIME type `application/vnd.mcp-ui.app+html` tells the host this is an MCP App, not a regular HTML resource.

How It Works: Step by Step

1. **User asks a question**
   "Show me keyword search results for 'mcp'"
2. **Host calls tools/list**
   Sees `key_word_search` has `_meta.ui.resourceUri` set. Preloads the UI resource.
3. **Host fetches resources/read ui://keyword-search/mcp-app.html**
   Server returns the HTML page. Host prepares the sandboxed iframe.
4. **LLM calls tools/call key_word_search**
   Server returns the matching file list as text content.
5. **Host renders HTML + pushes tool result**
   The iframe appears inline in the conversation. The app receives the tool result via `postMessage` and updates its UI.
6. **User clicks "Search Again"**
   The app calls back through `postMessage` → host routes → `tools/call` → fresh results → UI updates. No page reload, no context switch.

The Three New Java Types

UiMeta
Holds the `resourceUri` pointing to the HTML resource at the `ui://` URI.

AppMeta
Wraps `UiMeta` as the `_meta` field on a tool definition.

AppTool + AppToolBuilder
A tool record that includes `_meta` alongside name, description, and inputSchema. Fluent builder for construction.

Security Model — Sandboxed Iframes

- 🚫 **Cannot access parent DOM** — The app runs in a sandbox. It cannot read or modify the host page.
- 🚫 **Cannot read cookies** — No access to the host's cookies or local storage.
- 🚫 **Cannot navigate parent** — The app cannot redirect the conversation or navigate away.
- ✅ **Controlled postMessage** — All communication goes through a strict JSON-RPC postMessage channel managed by the host.

Hosts can safely render third-party MCP Apps without fully trusting the server author.

When to Use MCP Apps

✅ Good Fit

- Exploring complex data (tables, charts, maps)
- Configuring with many options (forms)
- Rich media (PDF viewers, image galleries)
- Real-time monitoring dashboards
- Multi-step workflows with approve/reject

❌ Not Needed

- Simple text responses
- Single-value lookups
- Linear workflows with no branching
- Cases where a list or table in markdown is sufficient

Supported By

✅ Claude (web) | ✅ Claude Desktop | ✅ VS Code Copilot | ✅ Goose | ✅ Postman | ✅ MCPJam

→ IDE DEMO — CHAPTER 6

Chapter 6 Live: The Keyword Search Dashboard

Switch to the IDE with `lessons/06-instructions.md` open as the script.

1. **Open the three spec records** in `src/main/java/com/workshop/mcp/spec/` — `UiMeta` (just a `resourceUri`), `AppMeta` (wraps it as `ui`), and `AppTool` (a Tool plus `_meta`). Three tiny records produce the entire `_meta.ui.resourceUri` structure the spec requires.
2. **Show** `AppToolBuilder` in `spec/builders/` — the same fluent pattern as every other builder, plus `withResourceUri("ui://keyword-search/mcp-app.html")`.
3. **Open** `lessons/mcp-app.html` **directly in the browser** — the standalone dashboard with mock data. Point at the comments explaining where `app.ontoolresult` and `app.callServerTool` take over in a real host.
4. **Walk the four** `IORouter` **changes:** the `io.modelcontextprotocol/apps` declaration in INITIALIZE, `AppToolsListResult` from TOOLS_LIST, the `ui://` resource added in RESOURCES_LIST, and the `ui://` branch in RESOURCES_READ serving the HTML with MIME type `text/html;profile=mcp-app`.
5. **In Inspector — Tools tab:** `key_word_search` now carries `_meta.ui.resourceUri`. **Resources tab:** `ui://keyword-search/mcp-app.html` appears; read it and show the MIME type.
6. **Apps tab:** click **Refresh Apps** — *Keyword Search App* appears. Run a search from inside the rendered dashboard and watch the tool round-trip happen from the iframe.

What the audience should observe: MCP Apps are just a tool plus a resource plus `_meta` — no new protocol machinery. Hosts that don't support Apps ignore `_meta` and the tool keeps working as text.

MCP Tasks — Call Now, Fetch Later

The 2025-11-25 utility for deferred execution, polling, and durable results

Chapter 7 | Experimental — 2025-11-25

The Problem Tasks Solve

The synchronous `tools/call` shape works fine for fast operations — sub-second searches, lookups, small computations. It falls apart for anything that takes longer than a polite roundtrip: large-scale processing, batch jobs, ML inference, external API calls with retries.

Tasks split a request into two trips: the receiver immediately acknowledges with a `taskId` and a status of `working`, runs the work in the background, and lets the requestor poll, retrieve, or cancel on its own schedule.

What Changes vs. Plain `tools/call`

Concern	Without Tasks	With Tasks
Long operations	Hold the connection open	Acknowledge fast, poll later
Crash recovery	Result lost when stream drops	Retrievable until TTL expires
Concurrent work	One slow call blocks the LLM	LLM dispatches and continues
Stateless HTTP	Needs SSE or webhooks	Just request/response polling

Capability Negotiation

Both sides advertise **which categories of request** may be task-augmented. Empty `{}` markers are intentional — presence is the signal.

Server declares (in `initialize` response)

```
{
  "capabilities": {
    "tasks": {
      "list": {},
      "cancel": {},
      "requests": {
        "tools": { "call": {} }
      }
    }
  }
}
```

Client declares (in `initialize` request)

```
{
  "capabilities": {
    "tasks": {
      "list": {},
      "cancel": {},
      "requests": {
        "sampling":    { "createMessage": {} },
        "elicitation": { "create": {} }
      }
    }
  }
}
```

Tool-Level Override

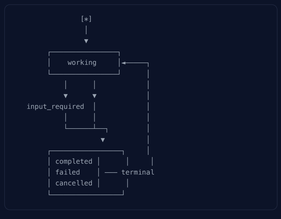
Each tool declares its own appetite via `execution.taskSupport` in the `tools/list` response:

```
{
  "name": "key_word_search",
  "execution": { "taskSupport": "optional" }
}
```

- `"required"` — client MUST task-augment
- `"optional"` — client MAY task-augment
- `"forbidden"` or absent — client MUST NOT

The Task Lifecycle

Tasks are state machines with three terminal states. Receivers are forbidden from moving a terminal task back into `working`.



Status values

`working`	Initial state. Receiver is processing.
`input_required`	Receiver needs more input before it can continue.
`completed`	Terminal — success. Result is retrievable.
`failed`	Terminal — underlying request failed.
`cancelled`	Terminal — cancelled by requestor.

Message Flow — Augmented `tools/call`

1. **Request augmentation**
   Client sends a regular `tools/call` with one extra field: `params.task = { ttl: 60000 }`.
2. **Immediate `CreateTaskResult`**
   Server replies in milliseconds: `{ task: { taskId, status: "working", ttl, pollInterval } }`. The underlying tool has not run yet.
3. **Background execution**
   Server spawns a thread (or hands off to a queue). The router keeps processing other requests while the task runs.
4. **Polling — `tasks/get`**
   Client polls at the server's suggested `pollInterval`. Each call returns the current `Task` snapshot without blocking.
5. **Retrieval — `tasks/result`**
   Once terminal (or if the client is willing to block), `tasks/result` returns exactly what the original `tools/call` would have returned, with `_meta.io.modelcontextprotocol/related-task` attached.
6. **Optional notification**
   Receivers MAY push `notifications/tasks/status` on transition. Requestors MUST NOT depend on it — polling stays the contract.

The Five New Methods

Method	Direction	Purpose
`tools/call` + `task`	req → rec	Implicit task creation via augmentation
`tasks/get`	req → rec	Non-blocking status poll
`tasks/result`	req → rec	Blocks until terminal, returns underlying result
`tasks/list`	req → rec	Cursor-paginated enumeration
`tasks/cancel`	req → rec	Best-effort cancel; `-32602` if terminal
`notifications/tasks/status`	rec → req	Optional status push

Related-Task Metadata

Every response or notification belonging to a task carries this key in its `_meta`:

```
"_meta": {
  "io.modelcontextprotocol/related-task": { "taskId": "786512e2-..." }
}
```

This is how a server-initiated `elicitation/create` (triggered *inside* a task) gets correlated back to its parent task — required for the `input_required` flow.

When *Not* to Use Tasks

Tasks add round trips. For short operations they cost more than they save. Skip task augmentation when:

- The call returns in under a second — the polling overhead dwarfs the work
- You're inside a tight loop — batch the calls instead
- You need ordered streaming output — tasks are point-in-time, not a stream
- Your transport already supports server-push (WebSockets, SSE) and you don't need durability

IDE DEMO — CHAPTER 7

Chapter 7 Live: Call Now, Fetch Later

Switch to the IDE with `lessons/07-instructions.md` open as the script.

1. **Tour the task records** in `src/main/java/com/workshop/mcp/spec/` — `Task`, `TaskStatus` (with `isTerminal()`), `TaskParams`, `CreateTaskResult`, and the per-method params/results. One `Task` shape is reused across get, cancel, list, and status notifications.
2. **Open `tasks/TaskStore.java`** — the in-memory lifecycle store: create as `working`, transition to `completed` / `failed` / `cancelled`, enforce terminal-state rules, expire by TTL.
3. **In `IORouter.java`**, show the capability negotiation — `tasks` in `ServerCapabilities` with `requests.tools.call`, and the tool advertising `execution.taskSupport = "optional"` in `tools/list`.
4. **Walk the augmented `TOOLS_CALL` branch** — when `params.task` is present: create the task, reply immediately with `CreateTaskResult`, and run `key_word_search` on a background thread (`runToolAsTask` sleeps ~4 seconds on purpose so the `working → completed` transition is observable in Inspector). The synchronous path is untouched.
5. **In Inspector:** first make a plain `tools/call` — results return synchronously, proving nothing broke. Then send a task-augmented call with `task: { "ttl": 60000 }` — a `CreateTaskResult` with `status: "working"` comes back in milliseconds.
6. **Poll the lifecycle:** `tasks/get` until `completed` (or catch the `notifications/tasks/status` push), then `tasks/result` to retrieve the search hits — note the `related-task` key in `_meta`.
7. **Finish with management:** `tasks/list` to enumerate, then start another augmented call and `tasks/cancel` it — show the terminal `cancelled` state and that a second cancel is rejected.

What the audience should observe: task augmentation is additive — the same tool, the same handler, one extra branch. The requestor decides per-call whether it wants sync or async semantics.

Supervised Agent — Claude Code Plugin

Connecting your MCP server to a live LLM with a composable plugin and a supervised 6-step audit workflow

Agent Chapter

From Inspector to Supervised Agent

You've built and tested the MCP server with the Inspector. Now you'll connect it to a **live LLM** — Claude Code — and extend it with a Claude Code plugin that bundles four composable skills into a supervised audit workflow.

Supervised Agent Pattern: Automate multi-step work but pause before the consequential output. You see all the evidence first — search results, structural analysis, source code reasoning — before committing to the final report.

Skill vs Plugin

Standalone Skill

A directory with a single `SKILL.md` file. Installed globally. Invoked as a single slash command.

```
~/.claude/skills/
└── keyword-search/      ← /keyword-search
    └── SKILL.md
```

Plugin

A directory with a `.claude-plugin/plugin.json` manifest and a `skills/` subdirectory. Bundles multiple skills under one namespace.

```
keyword-audit-plugin/
├── .claude-plugin/
│   └── plugin.json      ← metadata
└── skills/
    ├── keyword-search/
    │   └── SKILL.md     ← /plugin:keyword-search
    ├── keyword-report/
    │   └── SKILL.md     ← /plugin:keyword-report
    └── audit/
        └── SKILL.md     ← /plugin:audit
```

Aspect	Standalone Skill	Plugin
Structure	`name/SKILL.md`	`.claude-plugin/plugin.json` + `skills/`
Command	`/skill-name`	`/plugin-name:skill-name`
Install	Copy to `~/.claude/skills/`	`--plugin-dir` (dev) or `/plugin install`
Scope	One command	Many skills, one namespace

The Four Skills

DATA LAYER

`/keyword-audit-plugin:keyword-search <kw>`

keyword-search

Calls `key_word_search` MCP tool. Returns sorted file list with counts. The direct tool wrapper.

ANALYSIS LAYER

`/keyword-audit-plugin:keyword-report <kw>`

keyword-report

Groups results by Java package, identifies patterns, produces a markdown report with architectural recommendations.

EXECUTION LAYER

`/keyword-audit-plugin:java-script-runner --analyze <path>`

java-script-runner

The `FileAnalyzer.java` source is **embedded inside the SKILL.md**. Claude writes it to `/tmp`, runs it via JVM, displays the output, then cleans up. Completely self-contained.

ORCHESTRATOR

`/keyword-audit-plugin:audit`

audit

Ties all three into a supervised 6-step workflow with a checkpoint before the final report. The supervisor reviews all evidence before approving synthesis.

The Supervised Audit Workflow

1. **Keyword Search**
   For each configured keyword, calls `key_word_search` MCP tool → sorted file list with counts.
2. **Identify Hotspot File**
   Finds the file with the highest keyword count across all searches — the focal point of the audit.
3. **Java Structural Analysis**
   Calls `java-script-runner --analyze <hotspot-path>` → Claude writes the embedded FileAnalyzer.java source to `/tmp`, compiles and runs it via JVM, reports class/method structure.
4. **Read Source File**
   Claude reads the hotspot file directly — applies its own reasoning to understand *why* it has so many keyword matches.
5. **Package-Level Analysis**
   Runs `keyword-report` logic — groups all results by Java package, identifies architectural patterns and hotspots.
6. **Checkpoint — You Review All Evidence**
   The workflow pauses. You see: search results, structural profile, source code, and package analysis. **You decide** whether to approve synthesis or re-run with adjustments.
7. **Synthesis Report**
   Only after your approval — Claude synthesizes all evidence into a final architectural keyword audit report.

Three Layers Composing

Layer	Tool	Responsibility
Data	MCP `key_word_search`	Deterministic file search, counts — no interpretation
Execution	`java-script-runner` + JVM	Structural analysis via embedded Java — objective structural facts
Reasoning	Claude Code (Read + reasoning)	Reads source, understands context, explains *why it matters*

Development Mode — No Global Install

The `start_claude_mcp.sh` script starts Claude Code with both the MCP server and plugin loaded from the local project:

```
./start_claude_mcp.sh

# Internally runs:
claude --mcp-config .mcp-resolved.json --strict-mcp-config \
       --plugin-dir ./keyword-audit-plugin
```

- `--plugin-dir` loads the plugin live from source — edit any `SKILL.md`, restart, changes take effect immediately
- `.mcp-resolved.json` is generated on startup (expands `{YOUR_BASE}` to absolute path) and deleted on exit — you never edit it
- No global install required during development

IDE DEMO — AGENT CHAPTER

Agent Chapter Live: From Inspector to Supervised Agent

Switch to the terminal in the project root with `lessons/agent-instructions.md` open as the script.

1. **Show the wiring:** open `mcp.json` — the same JAR the Inspector has been driving all class, now registered with Claude Code. Then show the plugin layout: `keyword-audit-plugin/.claude-plugin/plugin.json` and the four skill folders under `skills/`.
2. **Skim two SKILL.md files:** `keyword-search` (thin wrapper over the MCP tool) and `audit` (the 6-step orchestrator with the approval checkpoint). Point out that `java-script-runner` embeds full Java 21 source inside its markdown.
3. **Launch:** `./start_claude_mcp.sh` — it builds the JAR if missing, expands the `mcp.json` template into `.mcp-resolved.json`, and starts Claude Code with `--plugin-dir`. **Pre-flight check:** the script reads the template from `lesson/mcp.json` (singular), but the repo ships it at `lessons/mcp.json` — before class, copy it to `lesson/mcp.json` (or symlink the folder) or the script exits with "No such file". Verify the plugin loaded with `/plugin` (Installed tab).
4. **Prove the raw MCP loop:** ask Claude *"What MCP tools are available?"* then *"Search for the keyword TODO — which file uses it most?"* — a live LLM now drives the same `key_word_search` tool.
5. **Run skills individually:** `/keyword-audit-plugin:keyword-search TODO` (data layer), `/keyword-audit-plugin:keyword-report TODO` (analysis layer), `/keyword-audit-plugin:java-script-runner --analyze src/main/java/com/workshop/mcp/IORouter.java` (execution layer — embedded FileAnalyzer runs via JVM).
6. **Run the full audit:** `/keyword-audit-plugin:audit` — searches run, the hotspot is identified and analyzed, Claude reads the source and reasons about *why*.
7. **The checkpoint:** the agent stops with *"Proceed to synthesis? (yes/no)"*. Pause here — this is the whole point. Review the evidence with the class, type `yes`, and show the final prioritized audit report.

What the audience should observe: the supervised pattern — five automated steps of real work, one human approval before the consequential output. Everything built across the previous chapters composes here: the MCP tool feeds skills, skills feed the orchestrator, and the human stays in the loop.

MCP Presentation – Runtime Facts

Key information about the MCP communication session and server capabilities — reference during IDE demos.

Runtime Facts — Pre-recorded Session Reference

Protocol Information

- **Protocol Version:** 2025-06-18
- **JSON-RPC Version:** 2.0
- **Transport:** STDIO
- **Communication:** Bidirectional

Client Details

- **Name:** mcp-inspector
- **Version:** 0.16.5
- **Role:** MCP Client
- **Session Duration:** ~2 min (17:37–17:39)

Server Details

- **Name:** agent-mcp-workshop
- **Version:** 0.0.1
- **Role:** MCP Server
- **Runtime:** JVM / Java 21

Client Capabilities

- **Sampling:** ✓
- **Elicitation:** ✓
- **Roots:** ✓ (listChanged: true)

Server Capabilities

- **Tools:** ✓ (listChanged: true)
- **Prompts:** ✓ (listChanged: true)
- **Resources:** ✓ (listChanged: false)
- **Completions:** ✓
- **Experimental:** elicitation + apps

Experimental Extensions

- `io.modelcontextprotocol/elicitation`
- `io.modelcontextprotocol/apps`

Available Tools

- **Tool Name:** `key_word_search`
- **Description:** Searches for keywords across project files
- **Parameters:** keyword *(the directory comes from `roots/list` or the elicitation form, not a tool arg)*
- **Task Support:** `execution.taskSupport: "optional"`
- **MCP App:** `ui://keyword-search/mcp-app.html`

Available Prompts

- **Prompt Name:** `search_keyword`
- **Description:** Creates a prompt to search for a word using the key_word_search tool
- **Arguments:** keyword (required)

Available Resources

- **Resource Count:** 70+ Javadoc files
- **Resource Type:** HTML documentation
- **Example:** `javadoc/com/workshop/mcp/spec/Capability.html`
- **Pagination:** ✓ (nextCursor: "pageNext")
- **UI Resource:** `ui://keyword-search/mcp-app.html`

Elicitation Details

- **Method:** `elicitation/create`
- **Request ID:** -4000
- **Message:** "Pick a directory for the keyword-search tool to search in"
- **Schema Fields:** directory (absolute path)
- **User Response:** directory = `/Users/.../some-repo`
- **Action:** accept → directory added to server's roots set

Session Flow

1. Initialization handshake
2. Elicitation — user input collection
3. Roots discovery
4. Ping & sampling
5. Resources discovery
6. Resource reading
7. Prompts discovery
8. Prompt execution
9. Tools discovery
10. Tool execution
11. Sampling response
12. Roots changes

Tool Execution Results

- **Search Keyword:** "mcp"
- **Files Found:** 58 files
- **Total Matches:** 400+ occurrences
- **Search Directory:** /Users/davidparry/code/github/agent-command

Reference this page during IDE demos to compare live Inspector output with the pre-recorded session data.

MCP Presentation – Trace Log

Raw JSON-RPC communication log — compare against your live Inspector output during demos.

Communication Trace

Toggle JSON Formatting | All | Received | Sent


```
[2026-06-09 09:49:06.310] [API][RECEIVED] {"jsonrpc":"2.0","id":0,"method":"initialize","params":{"protocolVersion":"2025-11-25","capabilities":{"sampling":{},"elicitation":{"form":{},"url":{}},"roots":{"listChanged":true},"tasks":{"list":{},"cancel":{},"requests":{"sampling":{"createMessage":{}},"elicitation":{"create":{}}}}},"clientInfo":{"name":"inspector-client","version":"0.22.0"}}}

[2026-06-09 09:49:06.314] [API][RECEIVED] initialize — client declared tasks capability, task-augmented requests enabled

[2026-06-09 09:49:06.316] [API][SENT]: {"jsonrpc":"2.0","id":0,"result":{"protocolVersion":"2025-11-25","capabilities":{"tools":{"listChanged":true},"prompts":{"listChanged":true},"resources":{"listChanged":false,"subscribe":false},"completions":{},"tasks":{"list":{},"cancel":{},"requests":{"tools":{"call":{}}}},"experimental":{"io.modelcontextprotocol/elicitation":{},"io.modelcontextprotocol/apps":{}}},"serverInfo":{"name":"agent-mcp-workshop","version":"0.0.1"}}}

[2026-06-09 09:49:06.320] [API][RECEIVED] {"jsonrpc":"2.0","method":"notifications/initialized"}

[2026-06-09 09:49:06.321] [API][SENT]: {"jsonrpc":"2.0","id":-1000,"method":"roots/list"}

[2026-06-09 09:49:06.340] [API][RECEIVED] {"jsonrpc":"2.0","id":-1000,"result":{"roots":[]}}

[2026-06-09 09:49:06.342] [API][RECEIVED] roots/list response — populated 0 root(s)

[2026-06-09 09:49:08.140] [API][RECEIVED] {"jsonrpc":"2.0","id":1,"method":"ping","params":{"_meta":{"progressToken":1}}}
```


Trace truncated for clarity. Real-world `key_word_search` results can list hundreds of files (e.g. when walking `node_modules`); the file dump is intentionally compact here so the protocol flow — synchronous call → lazy elicitation → task-augmented call → lifecycle — reads cleanly.

Color coding: Blue = Received (Client → Server) | Green = Sent (Server → Client) | Red = System events

DAVID PARRY

Principal Architect

x.com

YouTube

LinkedIn

Github

Discord

Email

https://github.com/David-Parry/agent-mcp-workshop/tree/complete